MMX

Lecture 12

Definition

- MMX (MultiMedia eXtension) is a set of instructions introduced by Intel to improve the performance of multimedia and data processing tasks, like image, audio, and video processing.
- It uses **SIMD (Single Instruction, Multiple Data)**, which means one instruction can process multiple pieces of data at once.

SIMD

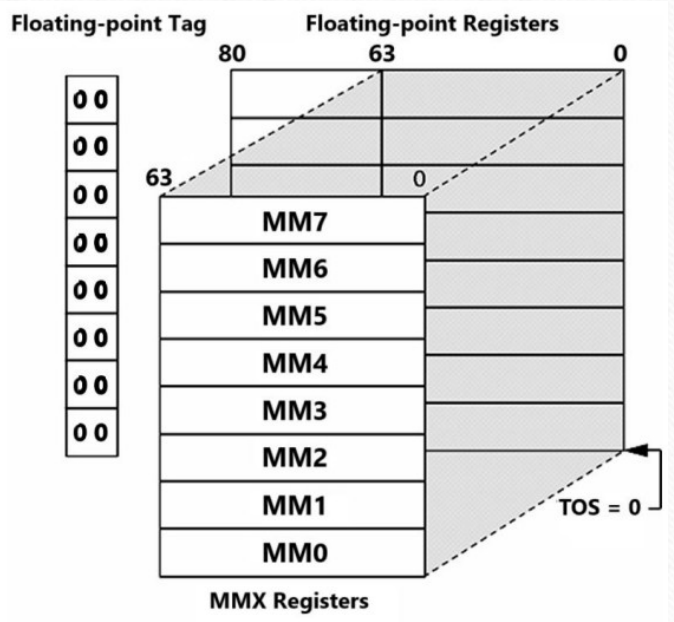
- Single Instruction works on Multiple Data
- Is there a way to manipulate array data with single instruction while working with General Purpose Registers (GPRs)?
  - No, we need indexing (offset + index; [BX+SI])
- Time consuming
- Less efficient
- Need a method that works on all elements of array parallelly
- Data must be *packed* into a vector

SIMD Registers

- GPRs only work on scalar (one value at a time) data
- They do not support SIMD
- SIMDS needs 64 bit storage space as registers.
- Makes use of Floating Point Unit (FPU) registers
- FPU is a part of CPU designated to perform operations on floating point data
- MMX uses these FP registers to manipulate integer data only

SIMD Registers

- Eight 80 bit registers, out which only least 64 bits are used for MMX
- MMX0 … MMX7
- Stacked together, keeping MMX0 on top
- Switching modes between FPU and MMX
  - EMMS instruction



Why SIMD for MMX

- MMX is used for image, audio or video processing
- Bulk data has to be processed like;
  - Processing pixels of an image
  - Applying filters or transformations to audio samples
  - Applying effects frame by frame on video samples
  - Matrix multiplication
  - Calculations for animations in games
- If GPRs are used, the processing will take a lot of time.

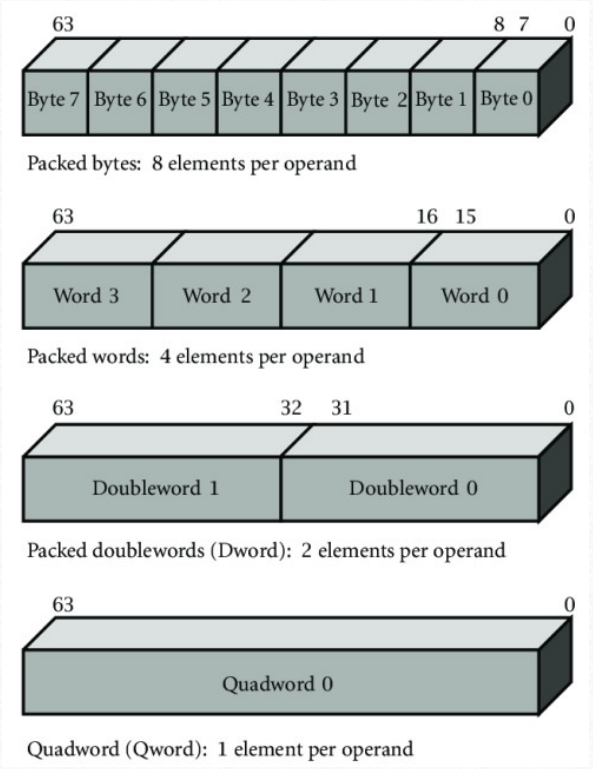
Key Points

- MMX operates on **64-bit registers** (reusing the floating-point registers).
- It works with **integer data types** (e.g., bytes, words, double words).
- Commonly used for operations like adding, multiplying, or shifting multiple data values simultaneously.
- It makes multimedia tasks faster by processing data in parallel instead of one at a time.

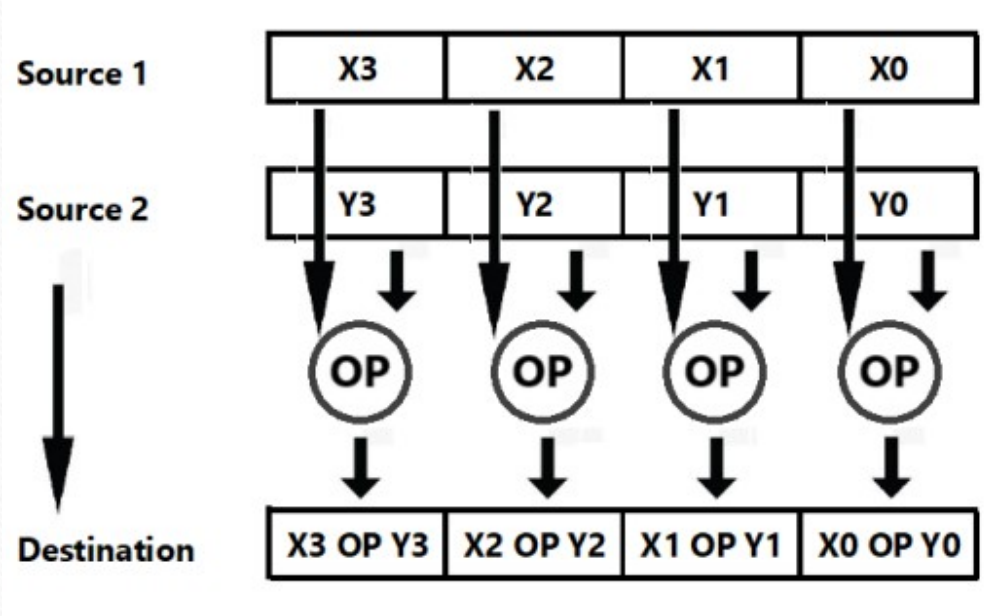
Limitations

- Works only on integer data, not FP
- Applies uniform operation of all data elements
- Cannot work on irregular data structures like linked lists, trees
- Performing division operation is challenging because of irregular pattern
- Performs well with data-parallel processes rather than task-parallel processes

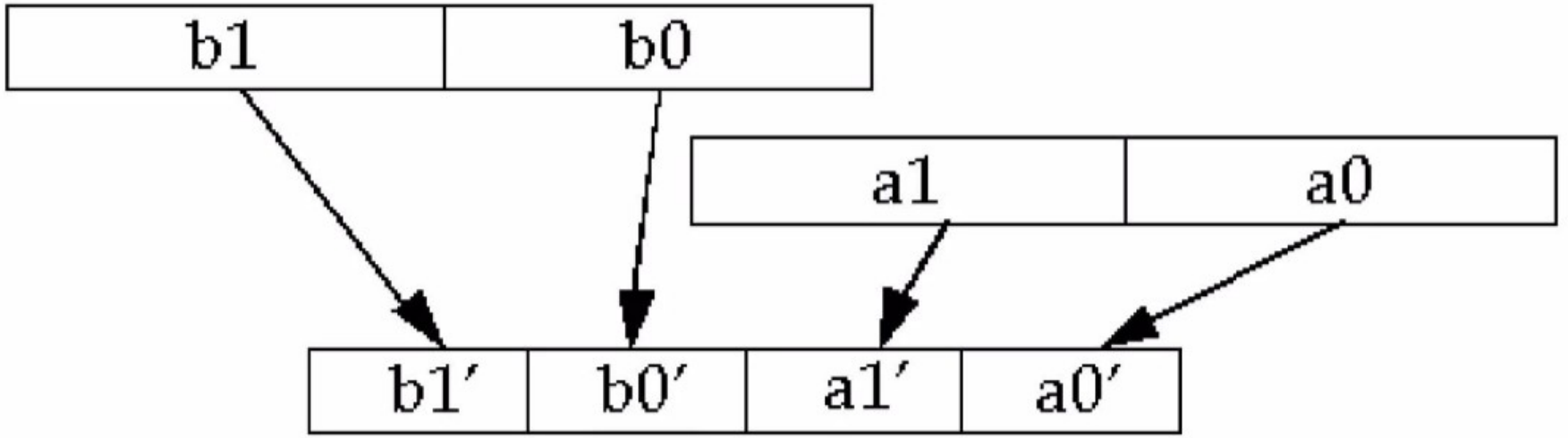
Organization of MMX Registers



MMX Operations



Pack: The PACK instructions in MMX are used to pack data elements from two MMX registers into a single MMX register



MMX Instruction Set

Category	Mnemonic	Number of Different OpCodes	Description
Arithmetic	PADD[B,W,D]	3	Add with wrap-around on [byte, word, doubleword]
	PADDS[B,W]	2	Add signed with saturation on [byte, word]
	PADDUS[B,W]	2	Add unsigned with saturation on [byte, word]
	PSUB[B,W,D]	3	Subtraction with wrap-around on [byte, word, doubleword]
	PSUBS[B,W]	2	Subtract signed with saturation on [byte, word]
	PSUBUS[B,W]	2	Subtract unsigned with saturation on [byte, word]
	PMULHW	1	Packed multiply high on words
	PMULLW	1	Packed multiply low on words
	PMADDWD	1	Packed multiply on words and add resulting pairs
Comparison	PCMPEQ[B,W,D]	3	Packed compare for equality [byte, word, doubleword]
	PCMPGT[B,W,D]	3	Packed compare greater than [byte, word, doubleword]
Conversion	PACKUSWB	1	Pack words into bytes (unsigned with saturation)
	PACKSS[WB,DWJ]	2	Pack [words into bytes, doublewords into words] (signed with saturation)
	PUNPCKH[BW,WD.DQ]	3	Unpack (interleave) high order [bytes, words, doublewords] from MMX register
	PUNPCKL[BW,WD.DQ]	3	Unpack (interleave) low order [bytes, words, doublewords) from MMX register
Logical	PAND	1	Bitwise AND
	PANDN	1	Bitwise AND NOT
	POR	1	Bitwise OR
	PXOR	1	Bitwise XOR
Shift	PSLL[W,D,Q]	6	Packed shift left logical [word, doubleword, quadword] by amount specified in MMX register or by immediate value
	PSRL[W,D,Q]	6	Packed shift right logical [word, doubleword, quadword] by amount specified in MMX register or by immediate value
	PSRA[W,D,Q]	4	Packed shift right arithmetic [word, doubleword] by amount specified in MMX register or by immediate value
Data Transfer	MOV[D,Q]	4	Move [doubleword, quadword] to MMX register or from MMX register
FP & MMX State Mgmt	EMMS	1	Empty MMX state

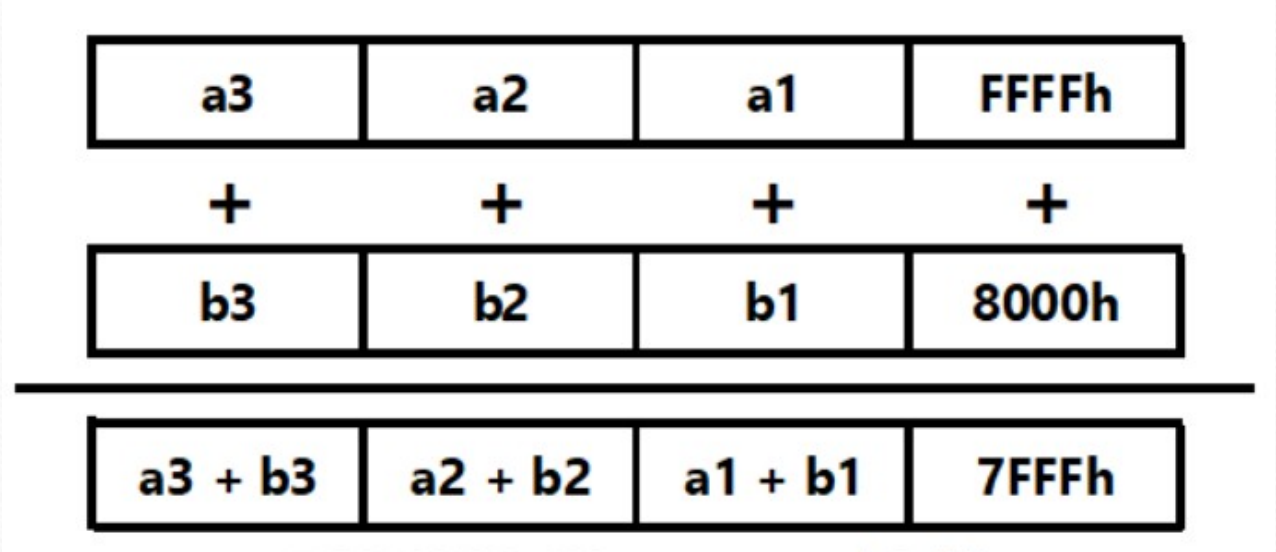
Wrap Around Add

- Suppose, we are dealing with 16 bit packed data
  - 4 slots of 16 bit data comprising 64 bits in total
- If sum exceeds 16 bits in any of the destination slot, or in other words, there is a carry on the most significant bit;
  - The carry (or the 17^{th} bit) will be discarded and the sum will be wrapped around to keep it to 16 bits
- Wrap-around: Adding anything to the max → goes back to min (like a clock).
- Saturation: Adding anything to the max → stays at max; subtracting from min → stays at min.

Wrap Around and Saturation

Bit-width	Type	Addition Behavior	Min	Max	Example (FF + 01)
8-bit	-	Wrap-around	00	FF	FF + 01 → 00
8-bit	Unsigned	Saturation	00	FF	FF + 01 → FF
8-bit	Signed	Saturation	80	7F	7F + 01 → 7F
16-bit	-	Wrap-around	0000	FFFF	FFFF + 0001 → 0000
16-bit	Unsigned	Saturation	0000	FFFF	FFFF + 0001 → FFFF
16-bit	Signed	Saturation	8000	7FFF	7FFF + 0001 → 7FFF
32-bit	-	Wrap-around	00000000	FFFFFFFF	FFFFFFFF + 1 → 00000000
32-bit	Unsigned	Saturation	00000000	FFFFFFFF	FFFFFFFF + 1 → FFFFFFFF
32-bit	Signed	Saturation	80000000	7FFFFFFF	7FFFFFFF + 1 → 7FFFFFFF

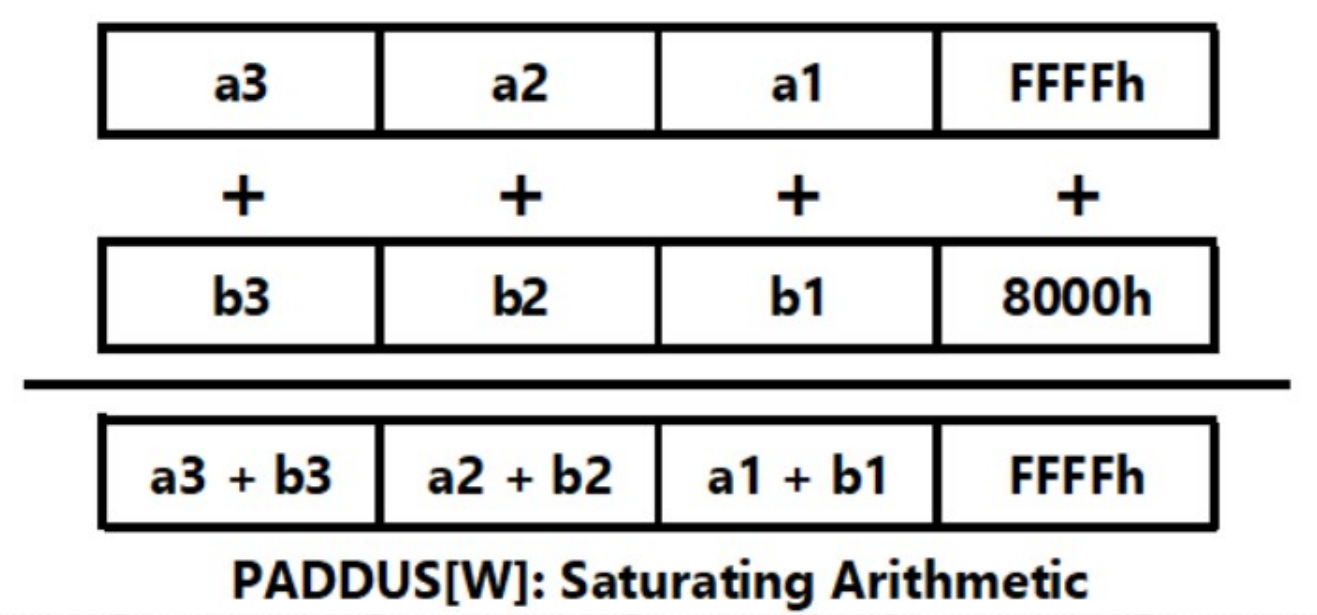
Signed Saturation



Unsigned Saturation

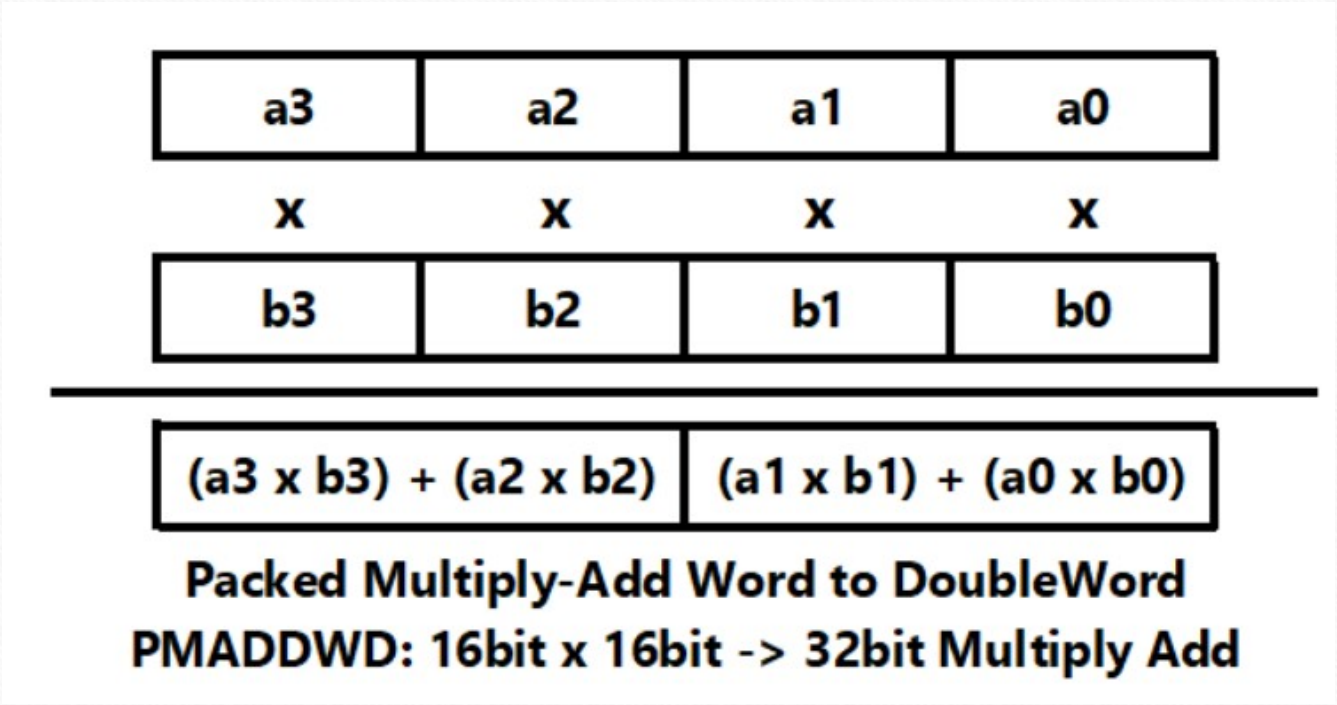
- In case of overflow or underflow (for subtraction), the result will be saturated/clamped to a specified value
- The range for 16 bit data is 0x0000 to 0xFFFF (0-65535 in decimal)
- If the result is lower than zero, it will be automatically converted (saturated) to 0x0000
- If the result exceeds 65535, it will be saturated to 0xFFFF
- For a signed word the largest and the smallest representable values are 7FFFh and 0x8000.

Unsigned Saturation

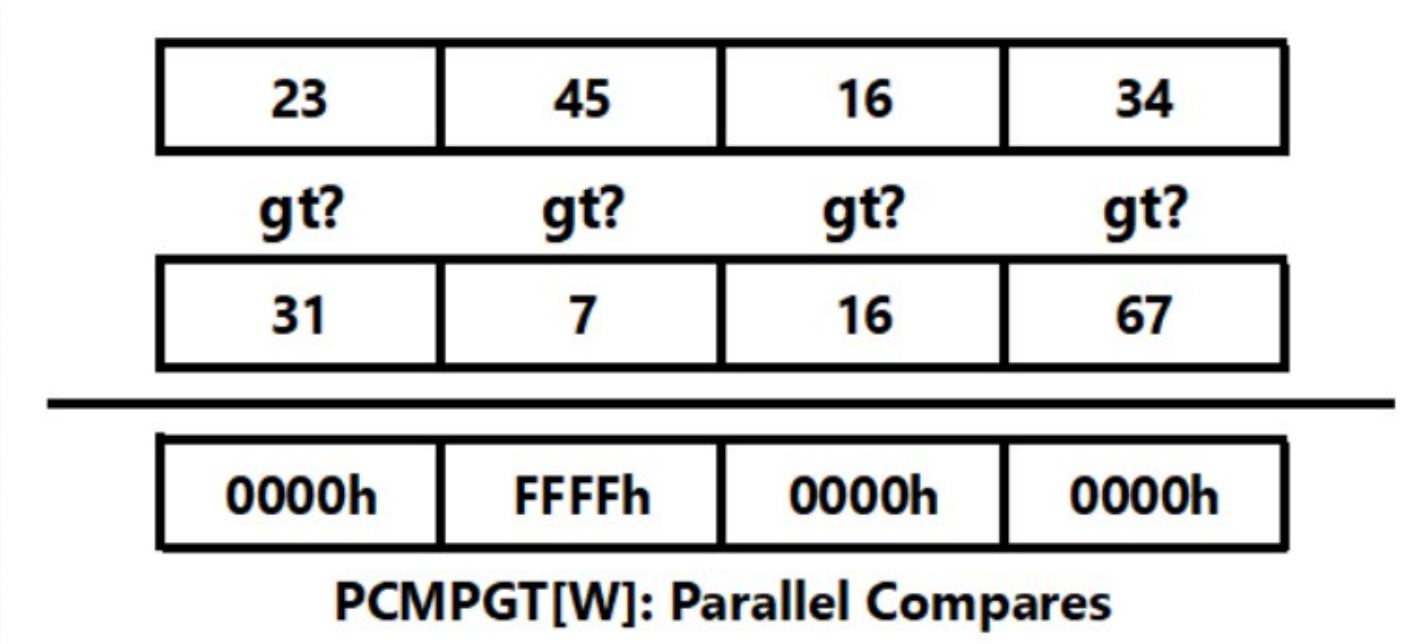


PADDUS[W]: Saturating Arithmetic

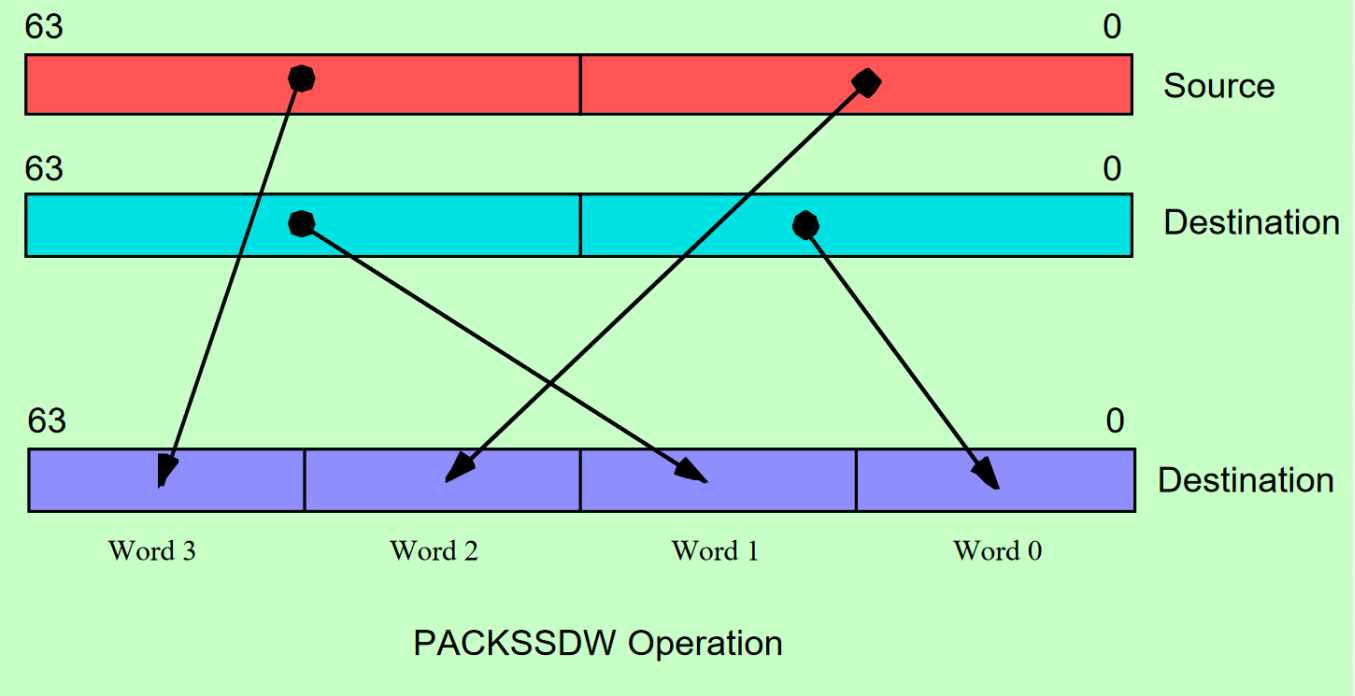
Multiplication using MMX



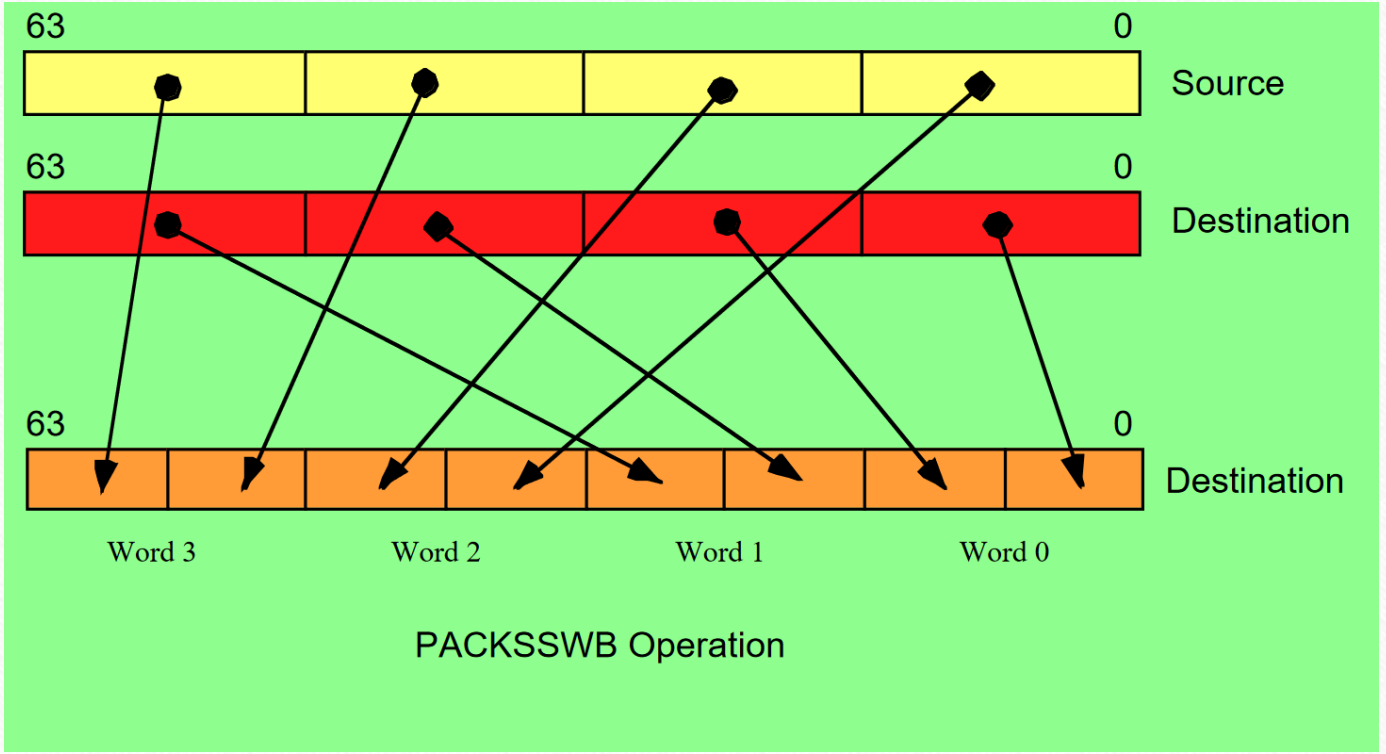
Comparison using MMX



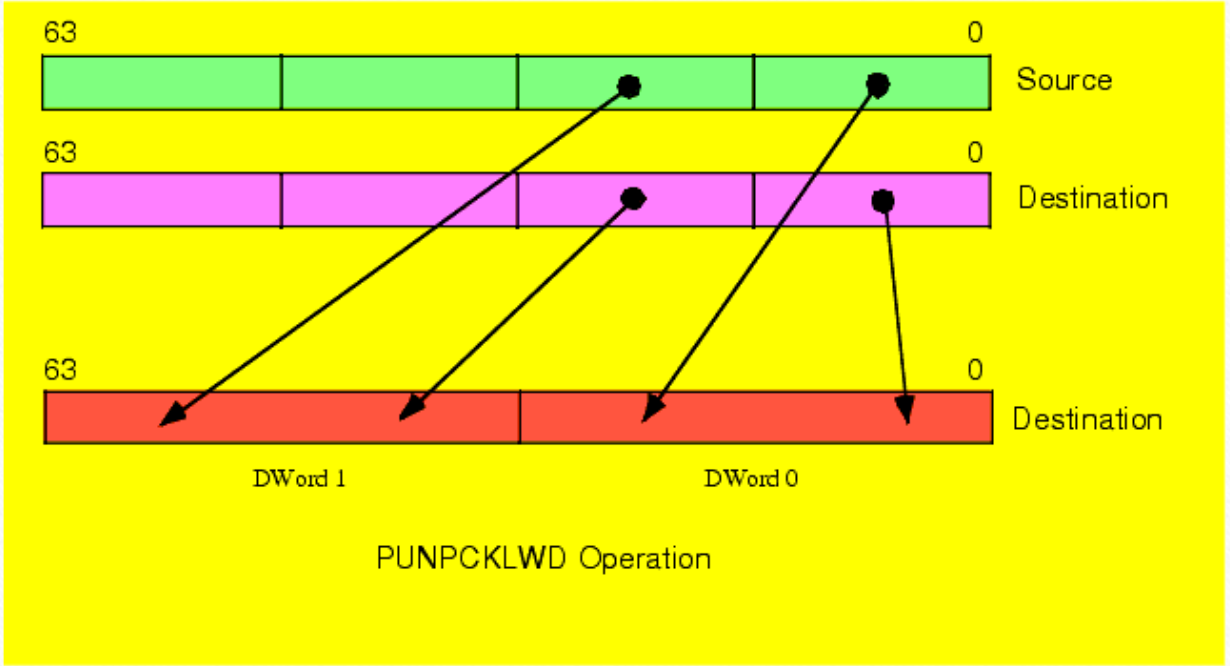
Double Word -> Word



Word -> Byte


63
0
Source
63
0
Destination
63
0
Destination
Word 3
Word 2
Word 1
Word 0
PACKSSWB Operation

Interleave – punpcklwd



EMMS Instruction

- Used to clear the multimedia state after performing MMX operations
- Used to ensure that the MMX state is not accidentally used in subsequent floating-point operations
- Used to reset the multimedia state of the processor, clearing any MMX registers and freeing them up for use by other instructions
- Used at the end of a block of code that uses MMX instructions to ensure that the processor’s multimedia state is in a known state before continuing to execute other instructions

Example – 1

```
#include<iostream>
using namespace std;
int main()
{
    const int size = 8;
    char array1[size] = { 12,2,3,4,5,6,7,8 };
    char array2[size] = { 1,7,8,9,10,11,12,13 };
    char array3[size];
    __asm
    {
        movq mm0, [array1]
        // move 8 bytes from address pointed by array1 to mm0 register
        movq mm1, [array2]
        // move 8 bytes from address pointed by array2 to mm1 register
        paddb mm0, mm1
        // add corresponding bytes of mm0 and mm1 and store result to mm0
        movq[array3], mm0
        // mov 8 bytes from mm0 to address pointed by array3
        emms // clearing multimedia state
    }
    for (int i = 0; i < size; i++)
    {
        cout << "Sum of " << (int)array1[i] << " and " << (int)array2[i]
            << " is " << (int)array3[i] << endl;
    }
    return 0;
}
```

Example – 2

```
#include<iostream>
using namespace std;
int main()
{
    const int size = 6;
    int array1[size] = { 1,2,3,4,5,6 };
    int array2[size];
    for (int i = 0; i < size * 4; i = i + 8)
        __asm
    {
        mov esi, i  // mov i to esi register
        movq mm0, [array1 + esi]    // moving 8 bytes from address pointed by array1 + esi
        pslld mm0, 1    // Shifting left every double word in mm0 register
        movq[array2 + esi], mm0     //moving contents of mm0 to memory pointed by array2 + esi
        emms        // clearing mmx state
    }
    for (int i = 0; i < size; i++)
    {
        cout << "Sum of " << (int)array2[i] << endl;
    }
    return 0;
}
```

Example – 3

```
#include<iostream>
using namespace std;
int main()
{
    const int size = 8;
    char array1[size] = { 25,50,80,110,125,127,30,90 };
    char array2[size] = { 100,100,100,100,100,100,100,100 };
    char array3[size];
    int sum = 0;
    __asm
    {
        movq mm0, [array1] // moving 8 bytes from array1 to mm0
        movq mm1, [array2] // moving 8 bytes from array2 to mm1
        pcmpgtb mm0, mm1
        /*/ 0xFF will replace bytes in mm0 that are greater than the
        corresponding bytes in mm1 and 0x00 will replace bytes in mm0 that
        are less than the corresponding bytes in mm1 /*/
        pand mm0, [array1]
        /*/ After bitwise and operation, mm0 will contain bytes that will
        be greater than 100 /*/
        movq[array3], mm0 // moving mm0 to array3
        emms
    }
    for (int i = 0; i < size; i++)
        sum += array3[i];
    cout << "Sum of numbers greater than 100 is: " << sum << endl;
    return 0;
```

Example – 4

```
#include<iostream>
using namespace std;
int main()
{
    const int size = 4;
    short int array1[] = { 2,3,4,5 };
    short int array2[] = { 1,2,3,4 };
    short int array3[size];
    __asm
    {
        movq mm0, [array1] // moving 8 bytes from array1 to mm0
        movq mm1, [array2] // moving 8 bytes from array2 to mm1
        pmullw mm0, mm1
        /*/ multiply corresponding word of mm0 and mm1 and store
        the lower word to mm0/*/
        movq[array3], mm0 //moving product to array3
        emms // clearing multimedia state
    }
    for (int i = 0; i < size; i++)
    {
        cout << "Product of " << (int)array1[i] << " and " <<
            (int)array2[i] << " is " << (int)array3[i] << endl;
    }
    return 0;
}
```